SIMPLIFIED CHEAT SHEET
for
8-bit AVR Programming in C

Literals			
Integer	Binary	0b11111111	0B11111111
	Octal	0377	
	Decimal	255	
	Hex	0xff	0xFF
	unsinged int	65535u	65535U
	long	12345678l	12345678L
	unsigned long	12345678ul	12345678UL
Floating Point	float	88.0f	88.123456F
	double	88.0	88.1234567
Character	char	'a'	'\n'
	String of char	"string"	"\0"

Data Types		
Type	Size (bits)	Domain
char	8	-2^7 to $2^7 - 1$
unsigned char	8	0 to $2^8 - 1$
int	16	-2^{15} to $2^{15} - 1$
unsigned int	16	0 to $2^{16} - 1$
long	32	-2^{31} to $2^{31} - 1$
unsigned long	32	0 to $2^{32} - 1$
float	32	$\pm 1.2\times10^{-38}$ to $\pm 3.4\times10^{38}$
double	32	same as float

Qualifiers	
const	Read only variable (compiler can optimize)
volatile	Unpredictable variable (compiler cannot optimize)

Variables	
Single	datatype identifier;
Initialized	datatype identifier = value;
Multiple	datatype identifier1, identifier2, identifier3;
Constant	const datatype identifier = value;

Identifiers	
Rules	Alphanumeric, no keywords, begin with a letter, less than 31 characters
Constants	CAPITALISED as a standard
Functions	Take form of a verb as a standard

Storage Classes	
register	Stored in a register instead of RAM, provides quick access, maximum size is register size
static	Retained value even when out of scope
extern	Global variable visible to ALL the program files

Enumeration	
Definition	enum enumName {value1, value2 };
Declaration	enum enumName variableName;
Access	variableName

Structures	
Definition	struct strctName{type member1; type member2;};
Declaration	struct strctName variableName;
	struct strctName *pointerName;
Initialization	struct strctName variableName = {value1, value2};
Access	variableName.member1
	pointerName->member1

Unions	
Definition	union unionName{type member1; type member2;}
Declaration	union unionName variableName;
Access	variableName.member1

Typecasting	
Typecast an expression of a type into another type	(type)expression

Type Definition	
Abbreviation for a type	typedef type abbreviation;
New type from a structure	typedef struct structName{type member1, type member2;}newTypeName;
New type from an enum	typedef enum enumName{value1, value2}newTypeName;

Pointers		
Declaration	type *pointerName;	
Access	Memory address	pointerName
	Value at memory address	*pointerName
	Address of operator	&
	Value at operator	*

Arrays		
Declaration	Uninitialized	type arrayName[size];
	Initialized with different values	type arrayName[size] = {value1, value2, value3};
	Initialized with same value	type arrayName[size] = {value};
	Compiler sets size	type arrayName[] = {value1, value2, value3};
Dimension	One	arrayName[size]
	Multi	arrayName[size1][size2]
Access	Value in element	arrayName[index]
		*(arrayName + index)
	Memory address	&arrayName[index]
		arrayName + index

Escape Characters	
Backslash	\\
Backspace	\b
Carriage return	\r
Double quote	\"
Horizontal tab	\t
Newline	\n
Null termination	\0
Question mark	\?
Single quote	\'
Vertical tab	\v

Functions		
Prototyping	type functionName([parameters...]);	
Declaration	type functionName([parameters...]){ [return value;]}	
Argument passing	By value	type function(type parameter); function(argument);
	By pointer	type function(type *pointerParameter); function(pointer);
		type function(type *pointerParameter); function(&argument);
Returning	return value;	
	return variable;	
	return &variable;	

Selection		
if	Single statement	if (expression) statement;
	Multiple statements	if (expression) {statement1; statement2;}
if ... else ...	if (expression) statement1; else statement2;	
if ... else if ...	if (expression1) statement1; else if (expression2) statement2; else statement3;	
Multiway	switch (expression) {case constant1: statement1; break; case constant2: statement2; break; default: statement3;}	

Iteration		
while	Single statement	while(condition) statement;
	Multiple statements	while(condition) {statement1; statement2;}
do ... while	Single statement	do statement; while(condition);
	Multiple statements	do {statement1; statement2;} while(condition);
for	Single statement	for(initialization; condition; increment/decrement) statement;
	Multiple statements	for(initialization; condition; increment/decrement) {statement1; statement2;}
Skip rest and jump to next iteration	continue;	
Skip rest and exit	break;	

Measurement	
Size of type	sizeof(type)
Size of array	sizeof(array)/sizeof(array[0])

Input/Output			
Header file	stdio.h		
Setup stream	FILE streamName = FDEV_SETUP_STREAM(writeFunction, readFunction, intent);		
Set as standard streams	stdin = &streamName;		
	stdout = &streamName;		
Character	getchar();		Returns a character from input stream
	putchar(char character);		Puts character to output stream
String	gets(char array);		Reads a line from input stream into char array
	puts("string");		Puts string of character to output stream
	puts(char array);		Puts contents of char array to output stream
Formatted data	scanf("format", addressOfArg1, pointerToArg2,...);		Reads value/s from input stream into argument/s
	printf("format", argument1, argument2, ...);		Prints values in argument/s to output stream
	Format specifier	%c	ASSCII character
		%d or %i	Integer
		%f or %F	Floating point
		%o	Octal
		%p	Pointer
		%s	String of char
		%u	Unsigned integer
		%x	Hex with x
		%X	Hex with X
		%%	% character
AVR	avr/io.h	I/O definitions	

Preprocessor Directives	
Standard header file	#include <filename.h>
	#include <path/filename.h>
Custom header file	#include "filename.h"
	#include "path/filename.h"
NAME with value	#define NAME value

Dynamic Memory		
Header file	stdlib.h	
Allocation	Variable	type *pointer = malloc(sizeof(type));
	Array	type *pointer = malloc(sizeof(type) * size);
	Structure	struct type pointer = malloc(sizeof(type));
Deallocation	free(void pointer);	
Reallocation	realloc(void pointer, int size);	

Unary Operators (descending precedence)	
Unary minus	-
Logical NOT	!
Bitwise NOT	~
Prefix increment	++expression
Prefix decrement	- - expression
Postfix increment	expression++
Postfix decrement	expression - -
Typecast	(type)expression
Value at	*
Address of	&
Size of	sizeof(type)

Binary Operators (descending precedence)	
Multiplication	*
Division	/
Modulus	%
Addition	+
Subtraction	-
Left shift	<<
Right shift	>>
Less than	<
Less than or equal	<=
Greater than	>
Greater than or equal	>=
Equality	==
Inequality	!=
Bitwise AND	&
Bitwise XOR	^
Bitwise OR	|
Logical AND	&&
Logical OR	||

Other Operators (descending precedence)	
Conditional ternary	condition ? expression1 : expression2;
Assignment	=
Multiplication and assignment	*=
Division and assignment	/=
Modulus and assignment	%=
Addition and assignment	+=
Subtraction and assignment	-=
Left shift and assignment	<<=
Right shift and assignment	>>=
Bitwise AND and assignment	&=
Bitwise XOR and assignment	^=
Bitwise OR and assignment	|=

Interrupts		
Header file	avr/interrupt.h	
Global interrupts	Set	sei();
	Clear	cli();
Interrupt Service Routine	ISR(interrupt_vector){statements;}	

Delay Library	
Header file	util/delay.h
Milliseconds	_delay_ms(double delay);
Microseconds	_delay_us(double delay);

Standard Library		
Header file	stdlib.h	
Generate random numbers	rand();	Returns a random integer between 0 and RAND_MAX
	srand(seed);	Seeds randomizer with an unsigned int
	RAND_MAX	0x7FFFFFFF
Convert a number to a string of ASSCII characters	itoa(int number, char array, int radix);	
	utoa(unsigned int number, char array, int radix);	
	ltoa(long number, char array, int radix);	
	ultoa(unsigned long number, char array, int radix);	
Convert a string of ASSCII char to a number	Float	atof(string);
	Integer	atoi(string);
	Long integer	atol(string);

Character Type Library	
Header file	ctype.h
Lowercase	tolower(char);
Uppercase	toupper(char);
Is of alphabet	isalpha(char)
Is lowercase of alphabet	islower(char)
Is uppercase of alphabet	isupper(char)
Is alphanumeric	isalnum (char)
Is digit	isdigit(char)
Is blank (' ' or '\t')	isblank(char)
Is whitespace (' ', '\f', '\n', '\r', '\t' or '\v')	isspace(char)

String Library	
Header file	string.h
Length excluding '\0'	strlen(string)
Copy string2 over string1	strcpy(string1, string2)
Concatenate strings	strcat(string1, string2)
Compare strings	strcmp(string1, string2)
Search for string2 in string1	strstr(string1, string2)

Math Library		
Header file	math.h	
Absolute value	Integer	abs(int number);
	Long	labs(long number);
	Float	fabs(double number);
Trigonometric	Cosine	cos(double angle);
	Sine	sin(double angle);
	Tangent	tan(double angle);
Exponential	exp(double exponent);	
Square root	sqrt(double number);	
Power	pow(double number, double power);	
Logarithmic	Natural	log(double number);
	Base 10	log10(double number);

Comments	
Single line	// single line comment
Multiline	/* multiline comments */

Dr. Thareendhra Keerthi Wijayasiriwardhane
University of Kelaniya, Sri Lanka